

The signal equation of MRI is given by

$$S_c(t) = \int_V s_c(\mathbf{r}) \rho(\mathbf{r}, t) e^{-i\gamma \int_0^t \Delta B(\mathbf{r}, \tau) d\tau} d\mathbf{r} \quad (1)$$

The difference ΔB from the main magnetic field B_0 can be expressed like

$$\Delta B(\mathbf{r}, t) = \Delta B_0(\mathbf{r}) + \sum_{\beta \in \{x, y, z\}} G_\beta^0(t) r_\beta + \sum_{\substack{|l|, |m| \geq 0 \\ \beta \in \{x, y, z\}}} G_\beta^{lm}(t) N_l^m(\mathbf{r}) \quad (2)$$

Where $N_l^m(\mathbf{r})$ are scalar fields describing the non-linear spatial variations of the magnetic field, typically described either by the solid harmonics

$$R_l^m(\mathbf{r}) = \|\mathbf{r}\|^l Y_l^m(\mathbf{r}), \text{ for spherical harmonics } Y_l^m(\mathbf{r}) \quad (3)$$

or by the

In the ideal case, the higher order terms from the harmonics are zero, and only the linear terms $G_\beta^0(t) r_\beta$ remain. However, in practice there are always some higher order terms present, and the ideal gradient fields prescribed are the actual gradient fields realized. The actual gradient waveforms can be modelled well by a convolution of the ideal gradient waveforms with an impulse response function $\alpha(t)$. This can be expressed for all orders of the gradient fields as

$$G_\beta^{lm}(t) = (\alpha_\beta^{lm} * G_\beta^{\text{ideal}})(t) \quad (4)$$

Or if gradient heating is to be taken into account, the impulse response function can be made time-varying as

$$G_\beta^{lm}(t) = (\alpha_\beta^{lm}(t) * G_\beta^{\text{ideal}})(t). \quad (5)$$

While the magnetization $\rho(\mathbf{r}, t)$ is time dependent, we consider readout to be short enough that the only time dependence is due to T_2 decay. Thus, we can express the magnetization as

$$S_c(t) = \int_V s_C(\mathbf{r}) \rho(\mathbf{r}) e^{-t/T_2(\mathbf{r})} e^{-i\gamma \int_0^t \Delta B(\mathbf{r}, \tau) d\tau} d\mathbf{r} \quad (6)$$

With this we can gather all terms that makes the signal equation deviate from a Fourier transform of the static magnetization in the linear gradient fields and form

$$S_c(t) = \int_V s_C(\mathbf{r}) \rho(\mathbf{r}) e^{\Phi(\mathbf{r}, t)} e^{-i\mathbf{k}(t) \cdot \mathbf{r}} d\mathbf{r} \quad (7)$$

where

$$\Phi(\mathbf{r}, t) = -t/T_2(\mathbf{r}) - i\gamma \int_0^t \Delta B_0(\mathbf{r}) d\tau + \sum_{\substack{|l|, |m| > 0 \\ \beta \in \{x, y, z\}}} N_l^m(\mathbf{r}) \int_0^t G_\beta^{lm}(\tau) d\tau \quad (8)$$

$$\mathbf{k}(t) = \gamma \int_0^t (G_x^0(\tau), G_y^0(\tau), G_z^0(\tau)) d\tau \quad (9)$$

Considering a voxel discretization, we can express the operators as matrices by

$$\begin{aligned} (\mathbf{A}_c)_{kj} &= s_C(\mathbf{r}_j) e^{\Phi(\mathbf{r}_j, t_k)} e^{-i\mathbf{k}(t_k) \cdot \mathbf{r}_j} \\ (\mathbf{A}_c^H)_{ik} &= s_C^*(\mathbf{r}_i) e^{\Phi^*(\mathbf{r}_i, t_k)} e^{i\mathbf{k}(t_k) \cdot \mathbf{r}_i} \\ (\mathbf{A}_c^H \mathbf{A}_c)_{ij} &= \sum_k s_C(\mathbf{r}_j) s_C^*(\mathbf{r}_i) e^{\Phi(\mathbf{r}_j, t_k) + \Phi^*(\mathbf{r}_i, t_k)} e^{-i\mathbf{k}(t_k) \cdot (\mathbf{r}_j - \mathbf{r}_i)} \end{aligned}$$

1 Approximations for fast application of the normal operator

To get back a Fourier transform, and to enable the Toeplitz property in $\mathbf{A}_c^H \mathbf{A}_c$. We introduce approximations

$$\exp(\Phi(\mathbf{r}_j, t_k)) \approx \sum_{l=0}^L \Omega_{kl} \Upsilon_{jl} \quad (10)$$

This enables us to approximate the forward and backward operators by evaluating L fourier transforms. We wish to get a similar decomposition for the normal operator. However, a naive application of the above approximation would give us a decomposition with L^2 Fourier transforms, which is infeasible. We thus try to get a decomposition into fewer Fourier terms. We begin with the naive multiplication

$$\exp(\Phi(\mathbf{r}_j, t_k) + \Phi^*(\mathbf{r}_i, t_k)) \approx \sum_{l_1=0}^L \sum_{l_2=0}^L (\Omega_{kl_1} \Omega_{kl_2}^*) (\Upsilon_{jl_1} \Upsilon_{il_2}^*) \quad (11)$$

1.1 Choosing decompositions

1.1.1 The exponential decomposition

Define $A_{ij}^k = \exp(\Phi(\mathbf{r}_j, t_k) + \Phi^*(\mathbf{r}_i, t_k)) = g_{jk} g_{ik}^*$. Where we further defined $g_{jk} := \exp(\Phi(\mathbf{r}_j, t_k))$. We seek a approximation of A_{ij}^k of the form

$$\hat{A}_{ij}^k = \sum_{p=0}^P \Lambda_{kp} \Theta_{jp} \Theta_{ip}^* \quad (12)$$

We use the usual Frobenius norm and seek to minimize the loss

$$\mathcal{L} = \sum_k \|A^k - \hat{A}^k\|_F^2 \quad (13)$$

Using Wirtinger calculus, we can compute the gradient with respect to Θ_{jp} as

$$\frac{\partial \mathcal{L}}{\partial \Theta_{jp}^*} = \sum_{k,i} \left(\sum_{p'=0}^P \Lambda_{kp'} \Theta_{jp'} \Theta_{ip'}^* - A_{ij}^k \right) \Lambda_{kp}^* \Theta_{ip} \quad (14)$$

and the gradient with respect to Λ_{kp} as

$$\frac{\partial \mathcal{L}}{\partial \Lambda_{kp}^*} = \sum_{j,i} \left(\sum_{p'=0}^P \Lambda_{kp'} \Theta_{jp'} \Theta_{ip'}^* - A_{ij}^k \right) \Theta_{jp} \Theta_{ip}^* \quad (15)$$

If we have orthogonality constraints on Θ_{jp} ,

$$\sum_j \Theta_{jp}^* \Theta_{jq} = \delta_{pq}, \quad (16)$$

then the gradients simplify.

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \Theta_{jp}^*} &= \sum_{k,i} \left(\sum_{p'=0}^P \Lambda_{kp'} \Theta_{jp'} \Theta_{ip'}^* - g_{jk} g_{ik}^* \right) \Lambda_{kp}^* \Theta_{ip} \\ &= \sum_k \sum_{p'=0}^P \Lambda_{kp'} \Lambda_{kp}^* \Theta_{jp'} \left(\sum_i \Theta_{ip'}^* \Theta_{ip} \right) - \sum_k g_{jk} \Lambda_{kp}^* \sum_i g_{ik}^* \Theta_{ip} \\ &= \sum_k \left[|\Lambda_{kp}|^2 \Theta_{jp} - g_{jk} \Lambda_{kp}^* \sum_i g_{ik}^* \Theta_{ip} \right] \end{aligned} \quad (17)$$

and

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial \Lambda_{kp}^*} &= \sum_{j,i} \left(\sum_{p'=0}^P \Lambda_{kp'} \Theta_{jp'} \Theta_{ip'}^* - g_{jk} g_{ik}^* \right) \Theta_{jp}^* \Theta_{ip} \\
&= \sum_{p'=0}^P \Lambda_{kp'} \left(\sum_{j,i} \Theta_{jp'} \Theta_{ip'}^* \Theta_{jp}^* \Theta_{ip} \right) - \sum_{j,i} g_{jk} g_{ik}^* \Theta_{jp}^* \Theta_{ip} \\
&= \sum_{p'=0}^P \Lambda_{kp'} \left(\sum_j \Theta_{jp'} \Theta_{jp}^* \right) \left(\sum_i \Theta_{ip'}^* \Theta_{ip} \right) - \sum_{j,i} g_{jk} g_{ik}^* \Theta_{jp}^* \Theta_{ip} \\
&= \sum_{p'=0}^P \Lambda_{kp'} \delta_{pp'} - \sum_{j,i} g_{jk} g_{ik}^* \Theta_{jp}^* \Theta_{ip} \\
&= \Lambda_{kp} - \left(\sum_j g_{jk} \Theta_{jp}^* \right) \left(\sum_i g_{ik}^* \Theta_{ip} \right) \\
&= \Lambda_{kp} - \left| \sum_i g_{ik} \Theta_{ip} \right|^2
\end{aligned} \tag{18}$$

Thus the optimal Λ_{kp} for fixed Θ_{jp} is given by

$$\Lambda_{kp} = \left| \sum_i g_{ik} \Theta_{ip} \right|^2. \tag{19}$$

Thus we arrive at the following alternating optimization scheme:

1. Initialize Θ_{jp} . If not initially orthogonal, orthogonalize.
2. Update $\Lambda_{kp} = \left| \sum_i g_{ik} \Theta_{ip} \right|^2$.
3. Update $\Theta_{jp} = \sum_k \Lambda_{kp} g_{jk} \sum_i g_{ik}^* \Theta_{ip}$, then orthogonalize.
4. Repeat steps 2 and 3 until convergence.

1.1.2 Decomposition of $\Omega_{kl_1} \Omega_{kl_2}^*$

For each k we have a $L \times L$ hermitian matrix, thus we can extract the $L(L+1)/2$ unique elements to form a $(L(L+1)/2) \times K$ matrix on which we can perform a SVD truncated to the P 'th largest singular values and reform the $L \times L$ matrix to get

$$\Omega_{kl_1} \Omega_{kl_2}^* \approx \sum_{p=0}^P C_{l_1 l_2, p} \Lambda_{kp}. \tag{20}$$

For each p we have a $L \times L$ hermitian matrix, for which we can perform a eigen-decomposition into the rank R approximation as

$$C_{l_1 l_2, p} \approx \sum_{r=0}^R \lambda_{rp} \hat{C}_{l_1 rp} \hat{C}_{l_2 rp}^*. \tag{21}$$

With $R = 1$ in (20) this would correspond to a Canonical Polyadic Decomposition.

Inserting these approximations into the normal operator (11) we get

$$\exp(\Phi(\mathbf{r}_j, t_k) + \Phi^*(\mathbf{r}_i, t_k)) \approx \sum_{l_1=0}^L \sum_{l_2=0}^L \sum_{p=0}^P \sum_{r=0}^R \Lambda_{kp} \lambda_{rp} \hat{C}_{l_1 rp} \hat{C}_{l_2 rp}^* \Upsilon_{jl_1} \Upsilon_{il_2}^* \quad (22)$$

$$= \sum_{p=0}^P \sum_{r=0}^R \Lambda_{kp} \lambda_{rp} \left(\sum_{l_1=0}^L \hat{C}_{l_1 rp} \Upsilon_{jl_1} \right) \left(\sum_{l_2=0}^L \hat{C}_{l_2 rp} \Upsilon_{il_2} \right)^* \quad (23)$$

$$= \sum_{p=0}^P \sum_{r=0}^R \Lambda_{kp} \lambda_{rp} \Theta_{jrp} \Theta_{irp}^* \quad (24)$$

1.1.3 Decomposition of $\Upsilon_{jl_1} \Upsilon_{il_2}^*$

Instead of decomposing the temporal factors $\Omega_{kl_1} \Omega_{kl_2}^*$, we may instead seek a decomposition of the spatial factors

$$\Upsilon_{jl_1} \Upsilon_{il_2}^*. \quad (25)$$

The goal is to obtain an approximation of the form

$$\Upsilon_{jl_1} \Upsilon_{il_2}^* \approx \sum_{p=0}^P C_{l_1 l_2, p} \Theta_{jp} \Theta_{ip}^*, \quad (26)$$

which would reduce the number of Fourier transforms required for application of the normal operator from L^2 to P .

Direct optimization of (26) is infeasible for realistic problem sizes, since the tensors $\Upsilon_{jl_1} \Upsilon_{il_2}^*$ cannot be explicitly formed or stored. Instead, we construct the decomposition using only matrix-vector products.

We begin by viewing Υ_{jl} as a matrix

$$\mathbf{\Upsilon} \in \mathbb{C}^{N \times L}, \quad (27)$$

where N is the number of spatial voxels. We then form the Hermitian operator

$$S_{ji} = \sum_{l=0}^L \Upsilon_{jl} \Upsilon_{il}^*. \quad (28)$$

This operator is never explicitly assembled. Instead, products $\mathbf{y} = \mathbf{S}\mathbf{x}$ are evaluated as

$$z_l = \sum_i \Upsilon_{il}^* x_i, \quad (29)$$

$$y_j = \sum_l \Upsilon_{jl} z_l. \quad (30)$$

Thus, application of $\mathbf{S}$ requires only multiplication by $\mathbf{\Upsilon}$ and $\mathbf{\Upsilon}^H$.

Using a Krylov subspace eigensolver such as Lanczos or randomized SVD, we compute the leading P eigenvectors of $\mathbf{S}$,

$$\sum_i S_{ji} \Theta_{ip} = \sigma_p^2 \Theta_{jp}, \quad (31)$$

with orthonormality

$$\sum_j \Theta_{jp}^* \Theta_{jq} = \delta_{pq}. \quad (32)$$

The vectors Θ_{jp} span the optimal rank- P subspace in the sense of the Eckart–Young theorem applied to $\mathbf{\Upsilon}$.

Projecting Υ_{jl} onto this basis gives

$$B_{lp} = \sum_j \Theta_{jp}^* \Upsilon_{jl}, \quad (33)$$

such that

$$\Upsilon_{jl} \approx \sum_{p=0}^P \Theta_{jp} B_{lp}. \quad (34)$$

Substituting into $\Upsilon_{jl_1} \Upsilon_{il_2}^*$ yields

$$\Upsilon_{jl_1} \Upsilon_{il_2}^* \approx \sum_{p=0}^P \sum_{q=0}^P \Theta_{jp} B_{l_1 p} B_{l_2 q}^* \Theta_{iq}^*. \quad (35)$$

The desired decomposition (26) is obtained by neglecting off-diagonal couplings $p \neq q$, giving

$$\Upsilon_{jl_1} \Upsilon_{il_2}^* \approx \sum_{p=0}^P B_{l_1 p} B_{l_2 p}^* \Theta_{jp} \Theta_{ip}^*. \quad (36)$$

We therefore identify

$$C_{l_1 l_2, p} = B_{l_1 p} B_{l_2 p}^*. \quad (37)$$

The quality of the approximation depends on the magnitude of the discarded cross terms $B_{l_1 p} B_{l_2 q}^*$ for $p \neq q$. To reduce these couplings, we may further rotate the basis vectors within the retained subspace. Let

$$\hat{\Theta}_{jp} = \sum_{q=0}^P \Theta_{jq} U_{qp}, \quad (38)$$

where $\mathbf{U} \in \mathbb{C}^{P \times P}$ is unitary. The projected coefficients become

$$\hat{B}_{lp} = \sum_q B_{lq} U_{qp}. \quad (39)$$

The rotation matrix $\mathbf{U}$ may then be chosen to minimize the off-diagonal energy

$$\mathcal{E}(\mathbf{U}) = \sum_l \sum_{p \neq q} |\hat{B}_{lp} \hat{B}_{lq}^*|^2. \quad (40)$$

Since this optimization is performed only over the small $P \times P$ unitary matrix $\mathbf{U}$, it remains computationally feasible even for large imaging problems. The optimization may be performed using Jacobi rotations, Riemannian gradient descent on the unitary group, or approximate joint diagonalization algorithms.

1.2 How to compute interpolants efficiently

Now, to find the approximations, performing the randomized SVD and SVD in these steps is computationally expensive, therefore, one can approach this with a histogram based method. First of all, the off-resonance and off-linear gradient terms only have effects on the signal on the support of the image. Thus, we do these approximations first only on voxels that are within the support. Next we can consider the feature vector

$$\overline{\Psi}(\mathbf{r}) = \begin{bmatrix} \mathbf{z}(\mathbf{r}) = \Delta B_0(\mathbf{r}) + 1/T_2(\mathbf{r}) \\ R_0^0(\mathbf{r}) \\ R_1^0(\mathbf{r}) \\ R_1^{-1}(\mathbf{r}) \\ \vdots \end{bmatrix} \quad (41)$$

Now, we can make a histogram of the feature vectors for all voxels in the support, and perform the approximations on the histogram bins instead of on the individual voxels.

Normal Operator via Toeplitz Embedding

Given the approximation (10), the discretised forward operator for coil c is

$$(\mathbf{A}_c \boldsymbol{\rho})_k = \sum_l \Omega_{kl} [\mathbf{F} \text{diag}(\Upsilon_l \odot s_c) \boldsymbol{\rho}]_k, \quad (42)$$

where $\mathbf{F}$ is the NUFFT and $(\Upsilon_l)_j = \Upsilon_{jl}$. The adjoint is

$$(\mathbf{A}_c^H \mathbf{s})_j = s_c^*(\mathbf{r}_j) \sum_l \Upsilon_{jl}^* [\mathbf{F}^H \text{diag}(\Omega_l^*) \mathbf{s}]_j. \quad (43)$$

Toeplitz structure

Define the PSF for k-space weights $w \in \mathbb{C}^K$ as the type-1 NUFFT

$$h_w = \mathbf{F}_{\text{adj}}(w), \quad (h_w)_n = \sum_k w_k e^{i\mathbf{k}(t_k) \cdot \mathbf{r}_n}. \quad (44)$$

The Toeplitz application uses a zero-padded FFT of size $2N$ in each dimension:

$$[\mathbf{F}^H \text{diag}(w) \mathbf{F} f] = h_w \star f = \text{IFFT}[\hat{H}_w \odot \text{FFT}(f_{\text{pad}})], \quad (45)$$

where $\hat{H}_w = \text{FFT}(h_{w,\text{pad}})$ is precomputed once.

L^2 Toeplitz approach

Using (11) directly:

$$(\mathbf{A}_c^H \mathbf{A}_c \boldsymbol{\rho})_i \approx \sum_{l_1=1}^L \sum_{l_2=1}^L \Upsilon_{il_2}^* (h_{l_1 l_2} \star (\Upsilon_{l_1} \odot s_c \odot \boldsymbol{\rho}))_i, \quad (46)$$

with one kernel per pair:

$$h_{l_1 l_2} = \mathbf{F}_{\text{adj}}(\Omega_{l_1} \odot \Omega_{l_2}^*). \quad (47)$$

Setup cost: L^2 type-1 NUFFTs. Application cost per iteration: L^2 Toeplitz multiplications (45).

$P \times R$ Toeplitz approach (reduced)

Inserting (24):

$$(\mathbf{A}_c^H \mathbf{A}_c \boldsymbol{\rho})_i \approx \sum_{p=1}^P \sum_{r=1}^R \Theta_{irp}^* (h_p \star (\Theta_{rp} \odot \boldsymbol{\rho}))_i, \quad (48)$$

where P distinct kernels (one per p):

$$h_p = \mathbf{F}_{\text{adj}}(\Lambda_p), \quad (49)$$

and the combined spatial weights:

$$\Theta_{jrp} = s_c(\mathbf{r}_j) \sum_{l=1}^L \hat{C}_{lrp} \Upsilon_{jl}. \quad (50)$$

Setup cost: P type-1 NUFFTs (independent of L). Application cost per iteration: $P \times R$ Toeplitz multiplications. With $R = 1$ this is P multiplications, and if $P \ll L^2$ we achieve a large speedup.

Algorithm (setup)

1. Compute $\Omega \in \mathbb{C}^{K \times L}$, $\Upsilon \in \mathbb{C}^{N_{\text{hist}} \times L}$ from SVD of e^Φ .
2. Form the $\frac{L(L+1)}{2} \times K$ matrix of unique upper-triangular products $\Omega_{kl_1} \Omega_{kl_2}^*$.
3. Truncated SVD of rank $P \Rightarrow \mathbf{\Lambda} \in \mathbb{C}^{K \times P}$, coefficient matrices $\mathbf{C}_p \in \mathbb{C}^{L \times L}$.
4. Rank- R SVD of each $\mathbf{C}_p \Rightarrow \hat{C}_{lrp}$.
5. $\Theta_{jrp} = \sum_l \hat{C}_{lrp} \Upsilon_{jl}$ scattered to full voxel grid.
6. P Toeplitz kernels: $\hat{H}_p = \text{FFT}[\mathbf{F}_{\text{adj}}(\Lambda_p)_{\text{pad}}]$.

Normal operator application

$$(\mathbf{A}^H \mathbf{A} \boldsymbol{\rho})_i = \sum_{p=1}^P \sum_{r=1}^R \Theta_{irp}^* \text{IFFT}[\hat{H}_p \odot \text{FFT}((\Theta_{rp} \odot \boldsymbol{\rho})_{\text{pad}})]_i. \quad (51)$$